Projects

PAGE NO.....
DATE / /

1) Expense_Tracker

- data
  - expense.txt
- src
  - main.py
  - expense_operations.py
  - file_handler.py

main.py

```
from expense_operations import add_expense, view_expense,
    get_total_expense, remove_expense
from file_handler import save_expenses_to_file,
                         load_expenses_from_file

# load expenses when the process
expenses = load_expenses_from_file()

def show_menu():
    print("\n Expense Tracker")
    print("1. Add Expense")
    print("2. View Expenses")
    print("3. Get Total Expenses")
    print("4. Remove Expense")
    print("5. Exit")

While True:
    show_menu()
    Choice = input("Choose an option: ")

    if choice == "1":
        add_expense(expenses)
    elif choice == "2":
        view_expenses(expenses)
```

```
elif choice == "3":
    get_total_expense(expenses)
elif choice == "4":
    remove_expense(expenses)
elif choice == "5":
    save_expenses_to_file(expenses)
    print("Expenses saved. Existing Expense Tracker.")
    break
else:
    print("Invalid option. Try again.")
```

Questions

1.) → `expenses = load_expenses_from_file()` calls the functions `load_expenses_from_file()`

→ The function returns a value (e.g. a list of expenses)

→ This return value is assigned to the variable `expenses`.

→ The variable `expenses` now holds the data returned by the function.

2) `\n` → Escape sequence (newline character).

```
print("Hello\nWorld")
```

output

```
Hello
World
```

PAGE NO.....
DATE / /

expense_operations.py:

```
def add_expenses(expenses):
    category = input("Enter Expense Category: ")
                .capitalize()
    amount = input("Enter Amount: ")
    date = input("Enter Date (YYYY-MM-DD): ")

    try:
        amount = float(amount)
        expense = {"category": category, "amount":
                   amount, "date": date}.
        expenses.append(expense).
    except ValueError:
        print("Invalid amount, Please enter a number.").

def view_expenses(expenses):
    if not expenses:
        print("No expenses recorded.")
    (1) return
    print("\n Expense List:").
    (2) { for index, expense in enumerate(expenses,
          start=1):
    (3) { print(f"{index}. {expense['date']}
          | {expense['category']} | ${expense
          ['amount']}")

def get_total_expenses(expenses):
    total = sum(expense["amount"] for expense in
    expenses)
    print(f"$ Total Expense: ${total:.2f}").

def remove_expense(expenses):
    view_expenses(expenses)
    try:
```

pop(index_no)

PAGE NO.....
DATE / /

④ expense_index = int(input("Enter expense number to remove: ")) - 1

if 0 <= expense_index < len(expenses):
    removed_expenses = expenses.pop(expense_index)
    print(f"Expense '{removed_expense['category']}' removed.").

⑤

else:
    print("Invalid expense number.").

except ValueError:
    print("Please enter a valid number.").

① what is the work of return in def view_expenses?

→ The return statement in the ~~view~~ ~~task(s)~~ view_expenses() is used to exit the function early if there are no expenses to display.

What Does return Do?

1) Exits the function!

→ When return is executed the function stops running immediately, and control is returned to the caller (the part of the code that called the function).

2) Optional Value

→ return can optionally return to the caller.

```
def add(a, b):
    return a+b
```

Here, returns sends the result of a+b back to the caller.

3) Early Exit

→ if a condition is met (e.g. no tasks are available), return can be used to exits the function early without executing the rest of the code.

PAGE NO.....
DATE / /

② What Does This line Do?

① enumerate(expenses, start=1)

→ The enumerate() function is used to loop over an iterable (like a list) while keeping track of both the index and the value of each time.

→ The start=1 argument tell enumerate() to start counting from 1 instead of the default 0.

② for index, expense in ....

→ This is a for loop that unpacks the result of enumerate() into two variable.

- index: The position of the current expense in the list (starting from 1).
- expense: The actual expense (a dictionary) at that position.

③ expenses

- This is the list of expenses being iterated over. Each item is the list ~~of~~ is a dictionary representing an expense.

How it works?

1. enumerate():
   - enumerate() takes an iterable (like a list) and returns a sequence of tuples, where each tuple contains:
     - An index (position) of the item.
     - The item itself.

e.g. if expenses is:

```
expenses = [
    {"date": "2023-10-01", "category": "Groceries",
     "amount": 50.0},
    {"date": "2023-10-01", "category": "Transport",
     "amount": 20.0}
```

PAGE NO.....
DATE / /

Then enumerate(expenses, start=1) will produce.

(1, {"date": "2023-10-01", "category": "Groceries", "amount": 50.03})

(2, {"date": "2023-10-02", "category": "Transport", "amount": 20.03})

2. Unpacking

-> The for index, expense in... syntax unpacks each tuple into two variables:

↳ index: The position of the expense (starting from 1)

↳ expense: The expense dictionary.

3. Loop Executions

-> For each ~~items~~ iterations of the loop:

↳ index will hold the current position (1, 2, 3, etc).

↳ expense will hold the current expense dictionary.

(3) The "|" (pipe symbol) is used as a separator to visually devide the different fields of the expense (date, category, and amount) in the output. It is not a special Python syntax; it's just a character chosen to make the output more readable and organized.

(4)

file_handler.py

```
import json

def save_expenses_to_file(expenses):
    with open("../data/expenses.txt", "w") as file
        json.dump(expenses, file)
    print("Expenses saved to file.")

def load_expenses_from_file():
    try:
        with open("../data/expenses.txt", "r") as file.
            return json.load(file)
    except(FileNotFoundError, json.JSONDecodeError):
        return []
```

1) Why there are two returns! ① return json.load (file) and ② return [] at last.

① return json.load (file) is for the success case (file exists and contains valid JSON).

② The second return (return []) is for the error case (file doesn't exist or contains invalid JSON). This ensures the function always returns a valid list, even if the file is missing or corrupted.

→ In these cases, the function cannot return the expected data, so it return an empty list [] as a default value.